

VLSI Internship Assignment on

**Design, UVM Verification and Synthesis of
a Universal Asynchronous Receiver
Transmitter (UART)**

Submitted by

Maresh Kumar Sahoo (2341019194)
B. Tech. (ECE) 7th Semester (Section-23414B2)



Organized by
CAREER DEVELOPMENT CELL
Institute of Technical Education and Research

SIKSHA 'O' ANUSANDHAN
DEEMED TO BE UNIVERSITY

Bhubaneswar, Odisha, India.
(July 202

Contents

Sl.No.	Title	Page No.
1	Introduction	1-3
2	Block Diagram	3-4
3	Modules & Codes	5-8
4	Testbench Code	9-13
5	Wave Forms	13-14
6	Synthesis Scripts & Reports	14-18
7	Static Timing Analysis Reports	19
8	Conclusion	20

1. **Introduction**

The Universal Asynchronous Receiver-Transmitter, commonly known as UART, is a fundamental hardware communication protocol used for serial data exchange between two devices. A primary advantage of UART is its simplicity, as it requires only two wires for bidirectional communication, one for transmitting (TX) and one for receiving (RX) eliminating the need for a separate, shared clock line. The transmitter and receiver operate using their own local clocks and agree on a predefined baud rate (communication speed). This makes the protocol highly reliable, easy to implement, and cost-effective for localized, point-to-point communication. Because of these benefits, UART is widely used across various embedded systems and everyday electronics. For instance, it is frequently utilized for debugging and logging, where a microcontroller like an Arduino sends console messages to a computer via a USB-to-UART bridge. Another common application is interfacing with GPS receivers, which continuously send continuous streams of location coordinates over a serial line to a host processor. Additionally, UART is heavily used to connect microcontrollers to wireless communication modules, such as Bluetooth HC-05 modules, enabling simple wireless serial data transfer between smart devices and sensors.

The working mechanism of UART is fundamentally asynchronous, meaning the transmitting and receiving devices do not share a common clock signal to synchronize their data transfer. Instead, both devices must be pre-configured to communicate at an identical baud rate, which defines the speed of transmission in bits per second, such as 9600 bps. To ensure the receiver knows exactly when a data transmission begins and ends, the actual data is encapsulated within a structured sequence known as a data frame. In an idle state, the transmission line is held at a high voltage level. When the transmitter is ready to send data, it initiates the frame with a start bit by pulling the line low for one clock cycle, signaling the receiver to begin listening. This is immediately followed by the data payload, which typically consists of eight data bits sent least significant bit first. After the data bits, an optional parity bit can be included for basic error checking, and finally, a stop bit pulls the line back to a high state to conclude the frame. Internally, a UART device consists of a baud rate generator to manage timing, a transmitter module to serialize parallel data into frames, and a receiver module to deserialize incoming frames back into parallel data. To maintain reliability despite minor clock drift between devices, modern receivers often employ a 16-times oversampling technique, reading the incoming signal sixteen times faster than the baud rate so they can capture the data exactly in the middle of a bit period and avoid false readings from electrical noise.

Baud Rate Generator Module (baud_rate_generator.v)

The Baud Rate Generator acts as the timing synchronizer for the entire UART protocol, ensuring that both transmitting and receiving operations are compatible to operate at a target speed, such as 9600 symbols per second. The Baudrate generator follows parametrized design approach which takes system clock frequency and required baudrate through parameter input and calculates the required states for counters to generate baudtick for Transmitter and Receiver modules.

First, it calculates the required counter states from input clock frequency and baudrate parameter values. Then it updates the counter states at positive clock edge until the required state is reached. Then it makes the baudtick high for exactly one clock period. To ensure receiver receives data at correct time without any error or glitch the receiver baudtick signal was oversampled 16 times.

Transmitter Module (Transmitter.v)

The Transmitter module converts parallel 8-bit input data into a serial bitstream. It operates using a 4-state Finite State Machine (FSM) governed by the baud_tick from the generator:

1. **Idle State (idle_state):** The module keeps the serial output line (Tx) high (1'b1). When the tx_start signal is triggered, it loads the 8-bit data_in into a register, resets its index counter to zero, and prepares to transmit.
2. **Start State (start_state):** On the next baud tick, it pulls the Tx line low (1'b0) to send the standard UART start bit, signaling the beginning of a frame.
3. **Data State (data_state):** Using a 3-bit counter (index), it iterates through the saved 8-bit data, sending out one bit at a time on the Tx line with every baud tick. Once the 8th bit (index 7) is sent, it transitions to the stop state.
4. **Stop State (stop_state):** It drives the Tx line back high (1'b1) to send the stop bit and immediately returns to the idle state.
5. **Busy Flag:** The module continuously outputs a busy signal that evaluates to 1'b1 whenever the FSM is outside of the idle_state, preventing new data from overwriting ongoing transmissions.

Receiver Module (Receiver.v)

The Receiver module captures serial data from the Rx line and converts it back into an 8-bit parallel output. It utilizes the 16x oversampling factor (driven by the receiver baud tick) to ensure robust glitch rejection. It uses a 5-state FSM:

1. **Idle State (idle_state):** It monitors the Rx line. If the line drops low during a baud tick, it suspects a start bit and moves to the start state, resetting its 4-bit sample counter.
2. **Start State (start_state):** The FSM waits until the sample counter reaches 7 (the midpoint of the 16-tick bit duration). If the Rx line is not still low at this midpoint, it assumes the

drop was a noise glitch and returns to idle. If valid, it waits until sample 15 to advance to the data state.

3. **Data State (data_state):** For each of the 8 data bits, it samples the Rx line exactly at the midpoint (sample == 7) and stores the read value into a temp_register. A 3-bit index tracks the bits up to 7 before proceeding to the stop state at sample 15 of the final bit.
4. **Stop State (stop_state):** It checks the Rx line at the midpoint (sample == 7) to verify it has returned high (1'b1), which represents the stop bit. Once verified, it transfers the assembled temp_register to the final data_out port.
5. **Done State (done_state):** The module briefly asserts the rx_done flag high for one tick to inform the rest of the system that new valid data is ready, then returns to idle.

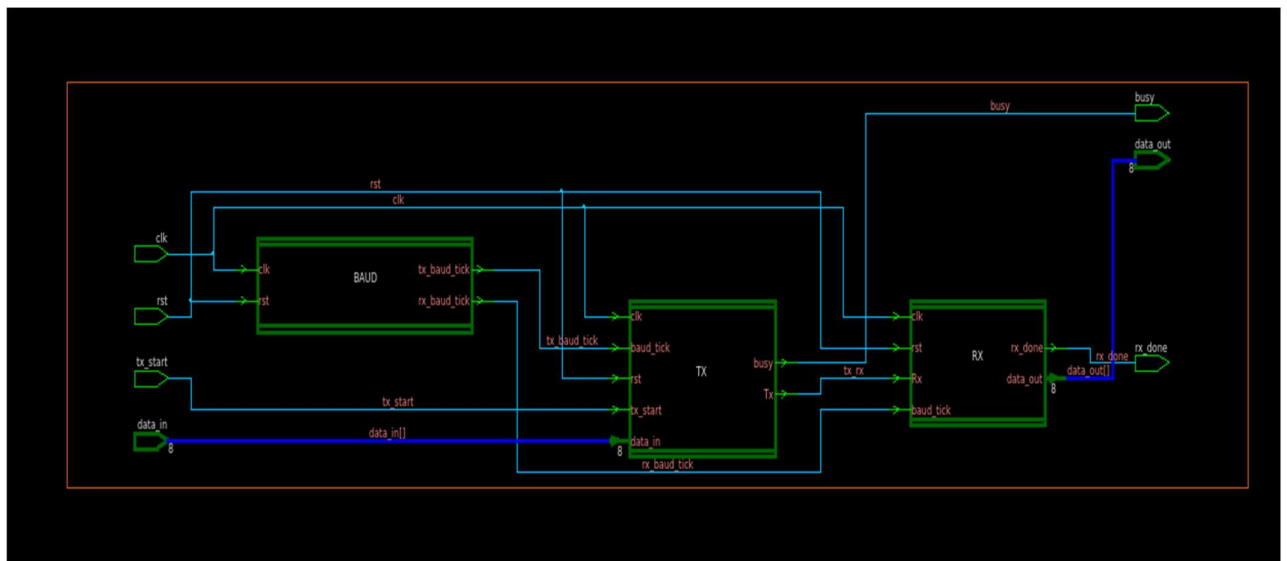
Top Module (top.v)

The Top module serves as the primary wrapper that defines the interconnections between the sub-modules.

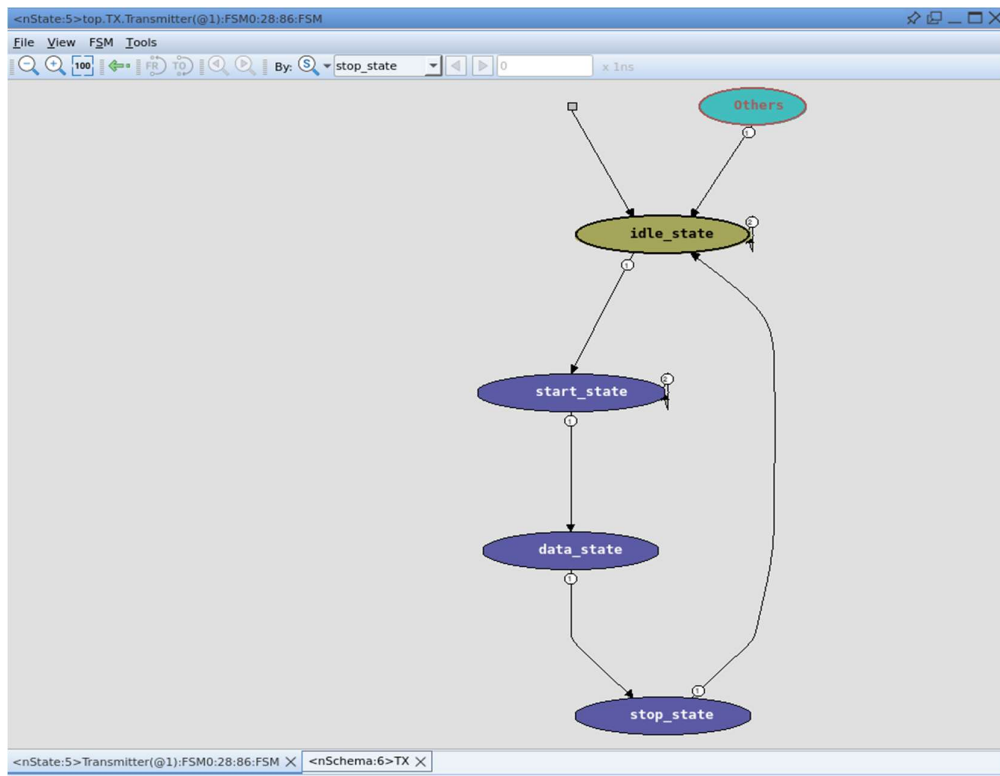
1. **Module Instantiation:** It brings together baud_rate_generator (as BAUD), Transmitter (as TX), and Receiver (as RX) into a single cohesive system.
2. **Internal Routing:** It utilizes internal wires, tx_baud_tick and rx_baud_tick, to distribute the proper timing signals from the baud generator to the respective communication modules.
3. **Loopback Connection:** Notably, this top module contains an internal wire named tx_rx that ties the output Tx pin of the transmitter directly into the input Rx pin of the receiver, establishing a localized loopback setup where data transmitted is immediately fed into the receiver.

2. Block Diagram

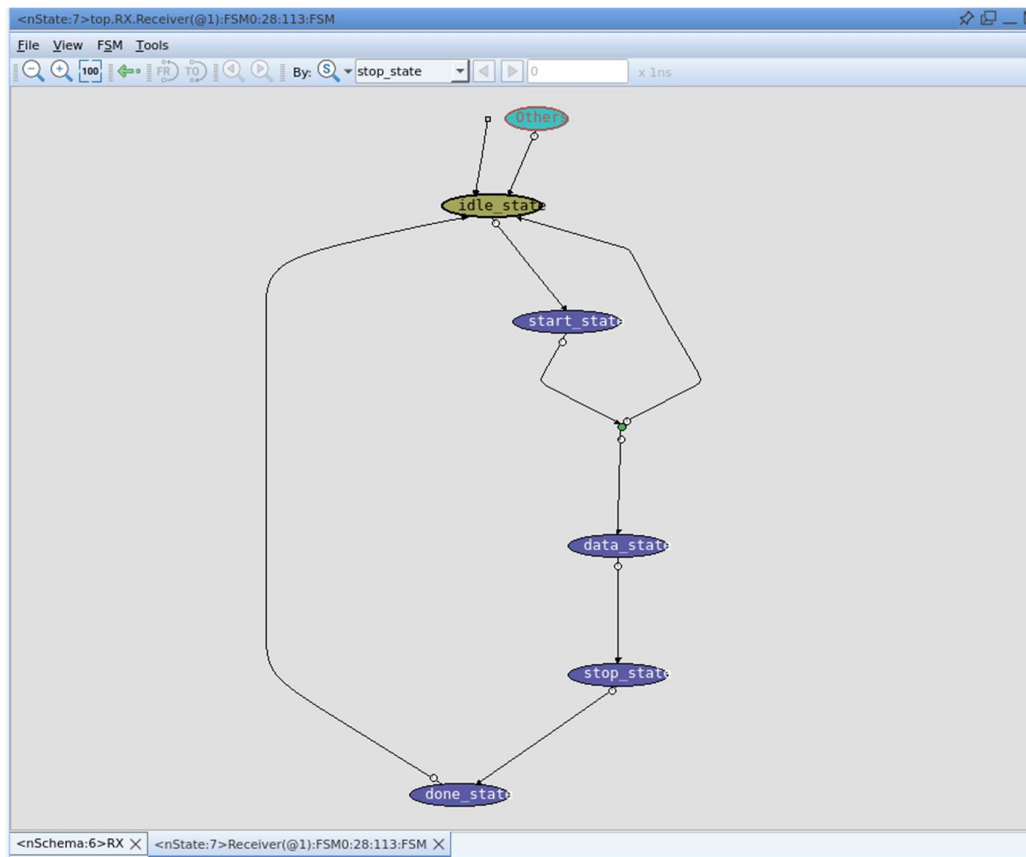
UART Block diagram/Architecture:



Transmitter FSM Diagram:



Receiver FSM Diagram:



3. Modules and codes:

UART Baud tick generator:

```
`timescale 1ns / 1ps
module baud_rate_generator#(parameter clk_frq = 100_000_000,parameter
baudrate = 9600)(
    input clk,rst,
    output tx_baud_tick, rx_baud_tick
);
    localparam tx_cnt_st = clk_frq / baudrate;
    localparam rx_cnt_st = clk_frq / (baudrate * 16); // 16 times
oversampled
    localparam tx_cnt_size = $clog2(tx_cnt_st);
    localparam rx_cnt_size = $clog2(rx_cnt_st);
    reg [tx_cnt_size-1:0] tx_counter;
    reg [rx_cnt_size-1:0] rx_counter;
    always @ (posedge clk)
        begin
            if (rst) begin
                tx_counter <= 0; // Initialize to 0 on reset
            end
            else begin
                if(tx_counter == tx_cnt_st)
                    tx_counter <= 0;
                else
                    tx_counter <= tx_counter + 1'b1;
            end
        end
    always @ (posedge clk)
        begin
            if (rst) begin
                rx_counter <= 0; // Initialize to 0 on reset
            end
            else begin
                if(rx_counter == rx_cnt_st)
                    rx_counter <= 0;
                else
                    rx_counter <= rx_counter + 1'b1;
            end
        end
    assign tx_baud_tick = (tx_counter == 0) ? 1'b1 : 1'b0;
    assign rx_baud_tick = (rx_counter == 0) ? 1'b1 : 1'b0;
endmodule
```

UART Transmitter

```
module Transmitter(
    input clk,baud_tick,rst,tx_start,
    input [7:0] data_in,
    output wire busy,
    output reg Tx
);
    parameter idle_state = 2'b00, start_state = 2'b01, data_state =
2'b10, stop_state = 2'b11;
    reg [7:0] data;
    reg [2:0] index;
    reg [1:0] state = idle_state;
    always @(posedge clk)
        begin
            if(rst) begin
```

```

        Tx <= 1'b1;
        state <= idle_state;
    end
    else begin
        case(state)
            idle_state:
                begin
                    Tx <= 1'b1;
                    if(tx_start)
                        begin
                            state <= start_state;
                            data <= data_in;
                            index <= 3'h0;
                        end
                    else
                        state <= idle_state;
                    end
                end
            start_state:
                begin
                    if(baud_tick)
                        begin
                            state <= data_state;
                            Tx <= 1'b0;
                        end
                    else
                        state <= start_state;
                    end
                end
            data_state :
                begin
                    if(baud_tick)
                        begin
                            Tx <= data[index];
                            if(index == 3'd7)
                                state <= stop_state;
                            else
                                begin
                                    index <= index + 3'd1;
                                end
                            end
                        end
                    end
                end
            stop_state :
                begin
                    if(baud_tick)
                        begin
                            Tx <= 1'b1;
                            state <= idle_state;
                        end
                    end
                end
            default : begin
                state <= idle_state;
                Tx <= 1'b1;
            end
        endcase
    end
    assign busy = (state != idle_state) ? 1'b1: 1'b0;
endmodule

```


UART Receiver

```
module Receiver(  
    input clk,rst,Rx,baud_tick,  
    output reg rx_done,  
    output reg [7:0] data_out  
);  
  
    parameter idle_state = 3'b000,start_state = 3'b001, data_state =  
    3'b011, stop_state = 3'b010, done_state = 3'b110;  
  
    reg [2:0] state = idle_state;  
    reg [3:0] sample = 0;  
    reg [2:0] index = 0;  
    reg [7:0] temp_register = 8'b0;  
    always @(posedge clk) begin  
        if(rst) begin  
            rx_done <= 0;  
            data_out <= 0;  
            state <= idle_state;  
            sample <= 0;  
            index <= 0;  
            temp_register <= 0;  
        end  
        else begin  
            case (state)  
                idle_state : begin  
                    if(Rx == 1'b0 && baud_tick)begin  
                        state <= start_state;  
                        sample <= 0;  
                    end  
                end  
                start_state : begin  
                    if(baud_tick) begin  
                        rx_done <= 0;  
                        if(sample == 7 && Rx != 1'b0)begin  
                            sample <= 0;  
                            state <= idle_state;  
                        end  
                        else if (sample == 15) begin  
                            state <= data_state;  
                            sample <= 0;  
                            index <= 0;  
                            temp_register <= 0;  
                        end  
                        else begin  
                            sample <= sample + 1'b1;  
                        end  
                    end  
                end  
                data_state : begin  
                    if(baud_tick) begin  
                        if (sample == 7) begin  
                            temp_register[index] <= Rx;  
                            sample <= sample + 1'b1;  
                        end  
                        else if(sample == 15) begin  
                            sample <= 0;  
                            if(index == 7) state <= stop_state;  
                            else index <= index + 1'b1;  
                        end  
                    end  
                end  
            end  
        end  
    end
```

```

        else begin
            sample <= sample + 1'b1;
        end
    end
end
stop_state : begin
    if(baud_tick) begin
        if(sample == 7 && Rx == 1'b1) begin
            state <= done_state;
            index <= 0;
            data_out <= temp_register;
        end
        else begin
            sample <= sample + 1'b1;
        end
    end
end
done_state : begin
    if(baud_tick) begin
        state <= idle_state;
        rx_done <= 1'b1;
        sample <= 0;
    end
end
default : begin
    state <= idle_state;
end
endcase
end
endmodule

```

UART Top module

```

module top(input clk,rst,tx_start,
    input [7:0] data_in,
    output busy,rx_done,
    output [7:0] data_out);
    wire tx_baud_tick,rx_baud_tick;
    wire tx_rx;
    baud_rate_generator BAUD( .clk(clk),
                                .rst(rst),
                                .tx_baud_tick(tx_baud_tick),
                                .rx_baud_tick(rx_baud_tick) );
    Transmitter TX (.clk(clk),
                    .rst(rst),
                    .tx_start(tx_start),
                    .baud_tick(tx_baud_tick),
                    .data_in(data_in),
                    .busy(busy),
                    .Tx(tx_rx) );
    Receiver RX(.clk(clk),
                .rst(rst),
                .baud_tick(rx_baud_tick),
                .rx_done(rx_done),
                .data_out(data_out),
                .Rx(tx_rx) );
endmodule

```

4. Testbench code

Buad Rate Generator Testbench:

```
module baudrate_gen_tb;
    reg clk,rst;
    wire tx_baudtick, rx_baudtick;
    baud_rate_generator uut(
        .clk(clk),
        .rst(rst),
        .tx_baud_tick(tx_baudtick),
        .rx_baud_tick(rx_baudtick)
    );
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end
    initial begin
        $dumpfile("baudrate.vcd");
        $dumpvars(0,uut);
        rst = 1;
        #10;
        rst = 0;
        #1000_000_000;
        $finish;
    end
    initial begin
        $monitor("At time = %t,tx_baudtick = %b,rx_baudtick",$time,
tx_baudtick ,rx_baudtick);
    end
endmodule
```

Transmitter Testbench:

```
module Tx_tb;
    reg clk,rst,baud_tick,tx_start;
    reg [7:0] data_in;
    wire busy, Tx;
    Transmitter dut(
        .clk(clk),
        .rst(rst),
        .baud_tick(baud_tick),
        .tx_start(tx_start),
        .data_in(data_in),
        .busy(busy),
        .Tx(Tx)
    );
    reg [31:0] baudtick_cnt;
    always @(posedge clk) begin
        if(rst) begin
            baud_tick <= 0;
            baudtick_cnt <= 0;
        end
        else if (baudtick_cnt == 10451) begin
            baud_tick <= 1;
            baudtick_cnt <= 0;
        end
        else begin
            baud_tick <= 0;
            baudtick_cnt <= baudtick_cnt + 1;
        end
    end
end
```

```

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
initial begin
    rst = 1;
    #10;
    rst = 0;
    sent_data(8'h00);
    $display("[time = %0t] sent A3", $time);
    sent_data(8'hFF);
    $display("[time = %0t] sent AA", $time);
    sent_data(8'h00);
    $display("[time = %0t] sent 00", $time);
    sent_data(8'hAA);
    $display("[time = %0t] sent FF", $time);
    sent_data(8'h3C);
    $display("[time = %0t] sent 3C", $time);
    #10000;
    $finish;
end
task sent_data(input [7:0] data_ip);
begin
    @(negedge clk);
    while (busy) @(negedge clk);
    data_in = data_ip;
    tx_start = 1;
    @(negedge clk);
    tx_start = 0;
    @(negedge clk);
    while (busy) @(negedge clk);
    #1000;
end
endtask
initial begin
    $monitor("Time = %0t ns | State/Tx Line = %b | Busy = %b | Din
= %h", $time, Tx, busy, data_in);
end
initial begin
    $dumpfile("TX.fst");
    $dumpvars(0, dut);
end
endmodule

```

Receiver Testbench:

```

module rx_tb;
reg clk, rst, baud_tick, rx;
wire [7:0] data_out;
wire rx_done;
Receiver dut(
    .clk(clk),
    .rst(rst),
    .baud_tick(baud_tick),
    .rx(rx),
    .rx_done(rx_done),
    .data_out(data_out)
);

reg [16:0] baudtick_cnt = 0;
always @(posedge clk) begin
    if(rst) begin

```

```

        baud_tick <= 0;
        baudtick_cnt <= 0;
    end
    else if (baudtick_cnt == 325) begin
        baud_tick <= 1;
        baudtick_cnt <= 0;
    end
    else begin
        baud_tick <= 0;
        baudtick_cnt <= baudtick_cnt + 1;
    end
end
initial begin
    clk = 0;
    forever #10 clk = ~clk;
end
initial begin
    //rst = 1;
    // #20;
    rst = 0;
    sent_serial_data(8'h00);
    $display("[time = %0t] sent 00", $time);
    sent_serial_data(8'hFF);
    $display("[time = %0t] sent FF", $time);
    sent_serial_data(8'h00);
    $display("[time = %0t] sent 00", $time);
    sent_serial_data(8'hAA);
    $display("[time = %0t] sent AA", $time);
    #1000;
    $finish;
end
// Task to transmit 1 full UART Frame (Start + 8 Data Bits + Stop Bit)
// 1 Bit Period = 16 * 326 * 20ns = 104,320 ns
task sent_serial_data(input [7:0] data_ip);
    integer idx;
    begin
        // Start bit
        rx = 1'b0;
        #(326 * 16 * 20);
        // Data bits (LSB First)
        for (idx = 0; idx < 8; idx = idx + 1) begin
            rx = data_ip[idx];
            #(326 * 16 * 20);
        end
        // Stop bit
        rx = 1'b1;
        #(326 * 16 * 20);
    end
endtask
initial begin
    $monitor("Time = %0t ns | data_out = %0h", $time, data_out);
end
endmodule

```

Top Module Testbench:

```

`timescale 1ns / 1ns
module UART_tb();
    reg clk, rst, tx_start;
    reg [7:0] data_in;
    wire busy, rx_done;

```

```

wire [7:0] data_out;
top dut (
    .clk(clk),
    .rst(rst),
    .tx_start(tx_start),
    .data_in(data_in),
    .busy(busy),
    .rx_done(rx_done),
    .data_out(data_out)
);
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
task sent_data(input [7:0] data_ip);
begin
    @(negedge clk);
    while (busy) @(negedge clk);
    data_in = data_ip;
    tx_start = 1;
    @(negedge clk);
    tx_start = 0;
    @(negedge clk); //waiting for the
previous data to be sent successfully
    while (busy) @(negedge clk);
end
endtask
initial begin
    rst = 1;
    #10;
    rst = 0;
    sent_data(8'h55);
    sent_data(8'hAA);
    sent_data(8'h00);
    sent_data(8'hFF);
    sent_data(8'hA5);
    sent_data(8'h3C);
    #1000;
    $finish;
end
initial begin
    $monitor("Time = %0t ns | Busy = %b | Data
sent      = %h | Data Received = %h", $time, busy,
data_in, data_out);
end
endmodule

```

UVM Code:

Top:

```
import uvm_pkg::*;
`include "uvm_macros.svh"

`include "uart_intf.sv"
`include "uart_item.sv"
`include "uart_seq.sv"
`include "uart_seqr.sv"
`include "uart_drv.sv"
`include "uart_mon.sv"
`include "uart_agt.sv"
`include "uart_sbd.sv"
`include "uart_env.sv"
`include "uart_test.sv"

module tb_top;
    logic clk,rst;
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
    uart_intf intf (
        .clk(clk),
        .rst(rst)
    );
    top dut(
        .clk(intf.clk),
        .rst(intf.rst),
        .tx_start(intf.tx_start),
        .data_in(intf.data_in),
        .busy(intf.busy),
        .rx_done(intf.rx_done),
        .data_out(intf.data_out)
    );
    initial begin
        uvm_config_db#(virtual uart_intf)::set(null,"*", "vif", intf);
        run_test("uart_test");
    end
    initial begin
        rst = 1;
        #20;
        rst = 0;
    end
endmodule
```

Interface:

```
interface uart_intf(input logic clk,rst);
    logic tx_start,busy,rx_done;
    logic [7:0] data_in,data_out;

    clocking drv_cb@(posedge clk);
        default input #1step output #1step;
        output tx_start;
        output data_in;
        input busy;
        input rx_done;
        input data_out;
    endclocking
endinterface
```

```

        clocking mon_cb @(posedge clk);
            default input #1step;
            input tx_start;
            input data_in;
            input busy;
            input rx_done;
            input data_out;
        endclocking

        modport DRIVER (clocking drv_cb,input clk,rst);
        modport MONITOR (clocking mon_cb,input clk,rst);
    endinterface

```

Item:

```

class uart_item extends uvm_sequence_item;
    bit clk,rst,tx_start;
    rand bit[7:0] data_in;
    bit busy,rx_done;
    bit [7:0] data_out;

    `uvm_object_utils_begin(uart_item)

        `uvm_field_int(clk,UVM_ALL_ON)
        `uvm_field_int(rst,UVM_ALL_ON)
        `uvm_field_int(tx_start,UVM_ALL_ON)
        `uvm_field_int(data_in,UVM_ALL_ON)
        `uvm_field_int(busy,UVM_ALL_ON)
        `uvm_field_int(rx_done,UVM_ALL_ON)
        `uvm_field_int(data_out,UVM_ALL_ON)

    `uvm_object_utils_end

    function new(string name = "uart_item");
        super.new(name);
    endfunction
endclass

```

Test:

```

class uart_test extends uvm_test;
    `uvm_component_utils(uart_test)
    uart_env env;

    function new(string name = "uart_test", uvm_component parent);
        super.new(name,parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = uart_env::type_id::create("env",this);
    endfunction

    task run_phase(uvm_phase phase);
        uart_seq seq;
        phase.raise_objection(this);
        seq = uart_seq::type_id::create("seq");
        seq.start(env.agt.seqr);
        #100;
        phase.drop_objection(this);
    endtask

endclass

```


Environment:

```
class uart_env extends uvm_env;
    `uvm_component_utils(uart_env)

    uart_agt agt;
    uart_sbd sbd;

    function new(string name = "uart_env", uvm_component parent);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agt = uart_agt::type_id::create("agt",this);
        sbd = uart_sbd::type_id::create("sbd",this);
    endfunction
    virtual function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);

        // Monitor -> Scoreboard
        agt.mon.ap.connect(sbd.exp_export);
    endfunction
endclass
```

Agent:

```
class uart_agt extends uvm_agent;
    `uvm_component_utils(uart_agt)
    uart_drv drv;
    uart_mon mon;
    uart_seqr seqr;
    function new(string name = "uart_agt", uvm_component parent);
        super.new(name,parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        drv = uart_drv::type_id::create("drv",this);
        mon = uart_mon::type_id::create("mon",this);
        seqr = uart_seqr::type_id::create("seqr",this);
    endfunction
    function void connect_phase(uvm_phase phase);
        drv.seq_item_port.connect(seqr.seq_item_export);
    endfunction
endclass
```

Sequence:

```
class uart_seq extends uvm_sequence;
    `uvm_object_utils(uart_seq)
    function new (string name = "uart_seq");
        super.new(name);
    endfunction
    task body();
        uart_item req;
        repeat(10) begin
            req = uart_item::type_id::create("req");
            start_item(req);
            assert(req.randomize());
            finish_item(req);
        end
    endtask
endclass
```

Sequencer:

```
class uart_seqr extends uvm_sequencer#(uart_item);
    `uvm_component_utils(uart_seqr)

    function new(string name = "uart_seqr", uvm_component parent);
        super.new(name, parent);
    endfunction

endclass
```

Driver:

```
class uart_drv extends uvm_driver#(uart_item);
    `uvm_component_utils(uart_drv)

    virtual uart_intf vif;
    function new(string name = "uart_drv", uvm_component parent);
        super.new(name, parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(virtual uart_intf)::get(this, "", "vif", vif))
            `uvm_fatal("DRV", "Interface Not Found")
    endfunction

    task run_phase(uvm_phase phase);
        uart_item req;
        forever begin
            seq_item_port.get_next_item(req);

            @(posedge vif.drv_cb);
            vif.drv_cb.tx_start <= 1'b1;
            vif.drv_cb.data_in <= req.data_in;

            @(posedge vif.drv_cb);
            vif.drv_cb.tx_start <= 1'b0;

            wait(vif.busy == 1'b1);
            wait(vif.busy == 1'b0);
            // @(posedge vif.drv_cb);

            seq_item_port.item_done();
        end
    endtask
endclass
```

Monitor:

```
class uart_mon extends uvm_monitor;
    `uvm_component_utils(uart_mon)

    virtual uart_intf vif;

    uvm_analysis_port#(uart_item) ap;

    function new(string name = "uart_mon", uvm_component parent);
        super.new(name, parent);
        ap = new("ap", this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
    endfunction
```

```

        if(!uvm_config_db#(virtual uart_intf)::get(this,"","vif",vif))
begin
        `uvm_fatal("MONITOR","Interface Not Found")
        end
    endfunction

    task run_phase(uvm_phase phase);
        uart_item item;
        logic [7:0] temp;
        forever begin

            item = uart_item::type_id::create("item");

            @(negedge vif.mon_cb.rx_done) begin
                item.data_in      = vif.mon_cb.data_in;
                item.rx_done      = vif.mon_cb.rx_done;
                item.data_out      = vif.mon_cb.data_out;
                item.tx_start      = vif.mon_cb.tx_start;
                //temp              = vif.mon_cb.data_in;
                item.busy          = vif.mon_cb.busy;
            end

            `uvm_info("MONITOR","Received Transaction",UVM_MEDIUM)
            item.print();
            ap.write(item);

        end
    endtask
endclass

```

Scoreboard

```

class uart_sbd extends uvm_scoreboard;
    `uvm_component_utils(uart_sbd)

    // TLM Analysis Exports to receive data from monitors
    uvm_analysis_export #(uart_item) exp_export;

    // TLM FIFOs to store incoming transactions until they can be compared
    uvm_tlm_analysis_fifo #(uart_item) exp_fifo;

    // Statistics counters
    int match_count;
    int mismatch_count;

    // Constructor
    function new(string name = "uart_sbd", uvm_component parent);
        super.new(name, parent);
        match_count = 0;
        mismatch_count = 0;
    endfunction

    // Build Phase: Instantiate the exports and FIFOs
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        exp_export = new("exp_export", this);
        exp_fifo   = new("exp_fifo", this);
    endfunction

    // Connect Phase: Connect the exports to the internal FIFOs
    virtual function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
    endfunction

```

```

    exp_export.connect(exp_fifo.analysis_export);
endfunction

// Run Phase: Continuously fetch and compare transactions
virtual task run_phase(uvm_phase phase);
    uart_item tr;
    uart_item prev_tr;

    super.run_phase(phase);

    forever begin

        exp_fifo.get(tr);

        if(prev_tr != null)begin
            if (prev_tr.data_in == tr.data_out) begin
                `uvm_info("SCBD_MATCH",$sformatf("Match! Expected: %0h, Actual:
%0h",prev_tr.data_in,tr.data_out), UVM_LOW)
                match_count++;
            end else begin
                `uvm_error("SCBD_MISMATCH",$sformatf("Mismatch! Expected: %0h,
Actual: %0h",prev_tr.data_in, tr.data_out))
                mismatch_count++;
            end

            end

            $cast(prev_tr,tr.clone());
        end
    endtask

//check for proper tx_start signal is feed into dut.(checked and corrected)
//drv is waiting for busy signal but i think drv cannot access the dut output
verify that(verified and now driver is synchronised)

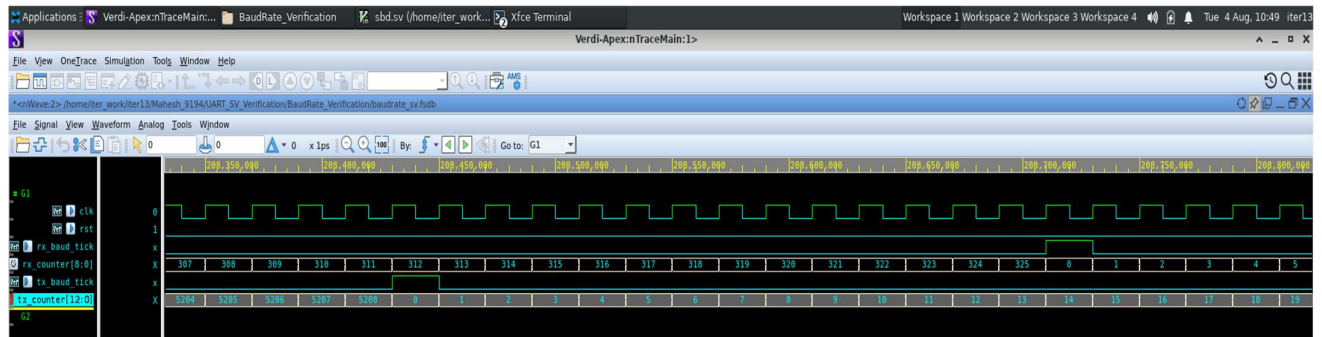
// Report Phase: Print final statistics at the end of the simulation
virtual function void report_phase(uvm_phase phase);
    super.report_phase(phase);
    `uvm_info("SCBD_REPORT", "-----",
UVM_NONE)
    `uvm_info("SCBD_REPORT", "          UART SCOREBOARD RESULTS          ",
UVM_NONE)
    `uvm_info("SCBD_REPORT", "-----",
UVM_NONE)
    `uvm_info("SCBD_REPORT", $sformatf("Total Test Cases Passed : %0d",
match_count), UVM_NONE)
    `uvm_info("SCBD_REPORT", $sformatf("Total Test Cases Failed : %0d",
mismatch_count), UVM_NONE)
    `uvm_info("SCBD_REPORT", "-----",
UVM_NONE)

    if (mismatch_count > 0) begin
        `uvm_error("SCBD_FAIL", "Test Failed: Data mismatches found.")
    end else if (match_count == 0) begin
        `uvm_warning("SCBD_EMPTY", "Test Warning: No data was compared.")
    end else begin
        `uvm_info("SCBD_PASS", "Test Passed: All data matched perfectly.",
UVM_NONE)
    end
endfunction

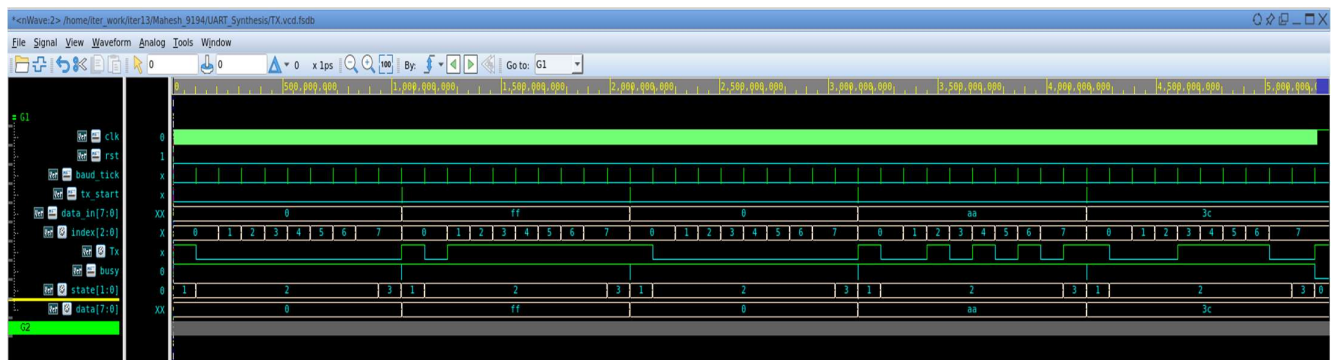
endclass

```

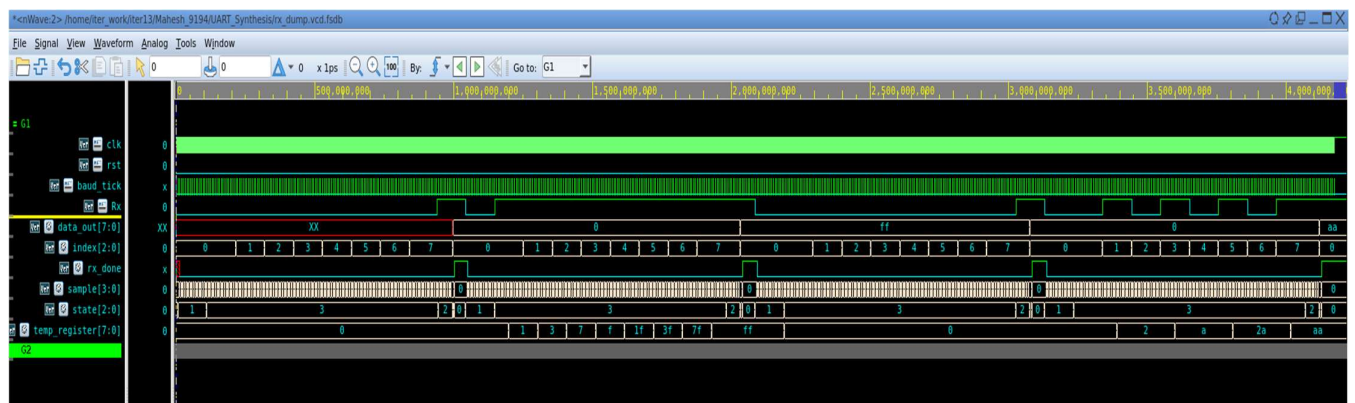
5. Wave forms



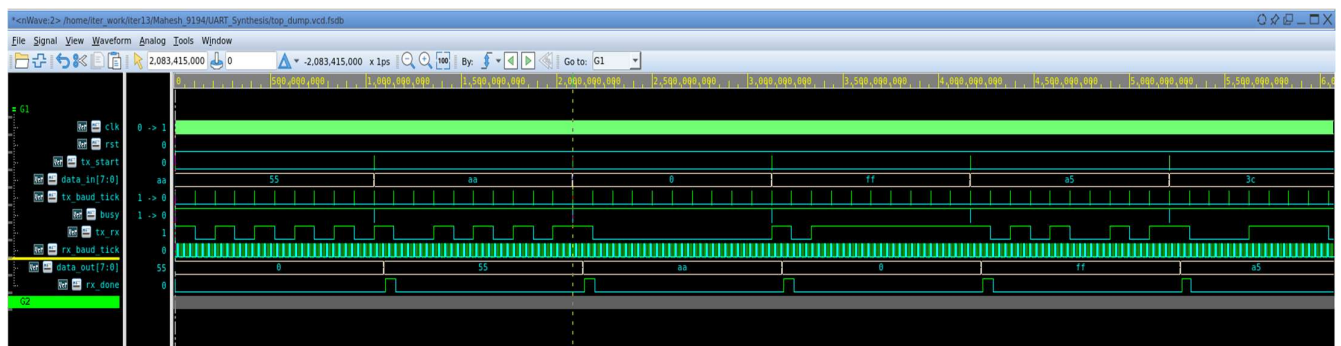
[Fig. 1: Buadrate Generator Output Waveform]



[Fig. 2: Transmitter Output Waveform]



[Fig. 3: Receiver Output Waveform]



[Fig. 4: UART (Top Module) Output Waveform]

6. Synthesis scripts and reports

TCL Script:

```
read_file -format ddc {/home/iter_work/iter13/Mahesh_9194/Synthesis/baudrate_work.ddc}

set link_library {*} /home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db}
set target_library {/home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db}

analyze -library WORK -format verilog /home/iter_work/iter13/Mahesh_9194/UART_Synthesis/top.v
analyze -library WORK -format verilog /home/iter_work/iter13/Mahesh_9194/UART_Synthesis/Transmitter.v
analyze -library WORK -format verilog /home/iter_work/iter13/Mahesh_9194/UART_Synthesis/Receiver.v
analyze -library WORK -format verilog /home/iter_work/iter13/Mahesh_9194/UART_Synthesis/baud_rate_generator.v

elaborate top

create_clock -name clk -period 10
create_clock -name clk -period 10 [get_ports clk]
set_input_delay 2.5 -clock clk [all_inputs]
set_output_delay 1 -clock clk [all_outputs]

link

set_driving_cell -lib_cell NBUFFX2_RVT [all_inputs]

compile

check_timing

report_timing
report_area
report_power
report_design
report_cell
report_qor
```

Area Report:

```
1
dc_shell> report_area

*****
Report : area
Design : top
Version: W-2024.09-SP3
Date   : Fri Aug 7 12:16:36 2026
*****

Library(s) Used:

    saed32rvt_tt0p85v25c (File: /home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db)

Number of ports:                100
Number of nets:                 445
Number of cells:                316
Number of combinational cells:  245
Number of sequential cells:     65
Number of macros/black boxes:   0
Number of buf/inv:              36
Number of references:           3

Combinational area:             531.923399
Buf/Inv area:                   45.745920
Noncombinational area:          430.011662
Macro/Black Box area:           0.000000
Net Interconnect area:          160.869217

Total cell area:                961.935061
Total area:                     1122.804278
1
```

Power Report:

```
*****
Report : power
        -analysis_effort low
Design : top
Version: W-2024.09-SP3
Date   : Fri Aug  7 12:17:17 2026
*****
```

Library(s) Used:

saed32rvt_tt0p85v25c (File: /home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db)

Operating Conditions: tt0p85v25c Library: saed32rvt_tt0p85v25c
Wire Load Model Mode: enclosed

Design	Wire Load Model	Library
top	8000	saed32rvt_tt0p85v25c
baud_rate_generator	8000	saed32rvt_tt0p85v25c
Transmitter	ForQA	saed32rvt_tt0p85v25c
Receiver	8000	saed32rvt_tt0p85v25c
baud_rate_generator_DW01_inc_0_DW01_inc_3	ForQA	saed32rvt_tt0p85v25c
baud_rate_generator_DW01_inc_1_DW01_inc_4	ForQA	saed32rvt_tt0p85v25c

Global Operating Voltage = 0.85
Power-specific unit information :
Voltage Units = 1V
Capacitance Units = 1.000000ff
Time Units = 1ns
Dynamic Power Units = 1uW (derived from V,C,T units)
Leakage Power Units = 1pW

Attributes

i - Including register clock pin internal power

Cell Internal Power = 30.2801 uW (98%)
Net Switching Power = 724.5558 nW (2%)

Total Dynamic Power = 31.0047 uW (100%)

Cell Leakage Power = 27.1983 uW

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs
io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)	
memory	0.0000	0.0000	0.0000	0.0000	(0.00%)	
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)	
clock_network	28.4282	0.0000	0.0000	28.4282	(48.84%)	i
register	0.7371	0.2128	1.6631e+07	17.5806	(30.21%)	
sequential	0.0000	0.0000	0.0000	0.0000	(0.00%)	
combinational	1.1148	0.5117	1.0568e+07	12.1941	(20.95%)	

Total 30.2801 uW 0.7246 uW 2.7198e+07 pW 58.2029 uW

1
dc_shell>

Cell Report:

```
Current design is 'top'.
dc_shell> get_cells
{BAUD TX RX}
dc_shell> report_cell
Information: Updating graph... (UID-83)

*****
Report : cell
Design : top
Version: W-2024.09-SP3
Date   : Mon Aug 10 10:16:03 2026
*****

Attributes:
  b - black box (unknown)
  h - hierarchical
  n - noncombinational
  r - removable
  u - contains unmapped logic

Cell          Reference      Library      Area  Attributes
-----
BAUD          baud_rate_generator  332.674506   h, n
RX            Receiver            428.740936   h, n
TX            Transmitter         200.519619   h, n
-----
Total 3 cells          961.935061
1
dc_shell> report_design
Information: Updating design information... (UID-85)

*****
Report : design

Log      History
dc_shell> |
```

Design Report:

```
dc_shell> report_design
Information: Updating design information... (UID-85)

*****
Report : design
Design : top
Version: W-2024.09-SP3
Date   : Mon Aug 10 10:16:22 2026
*****

Design allows ideal nets on clock nets.

Library(s) Used:

  saed32rvt_tt0p85v25c (File: /home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db)

Local Link Library:

  {/home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db}

Flip-Flop Types:

  No flip-flop types specified.

Latch Types:

  No latch types specified.

Operating Conditions:

  Operating Condition Name : tt0p85v25c
  Library : saed32rvt_tt0p85v25c
  Process : 1.00
  Temperature : 25.00
  Voltage : 0.85
  Interconnect Model : balanced tree
```



```
Wire Loading Model:

    Selected automatically from the total cell area.

Name      : 8000
Location   : saed32rvvt_tt0p85v25c
Resistance : 0.0015727
Capacitance : 0.000312
Area       : 0.01
Slope      : 90.6464
Fanout     Length  Points Average Cap Std Deviation
-----
1      13.94
2      31.80
3      51.61
4      73.61
5      98.05
6     125.17
7     155.23
8     188.46
9     225.12
10    265.45
11    309.71
12    358.13
13    410.96
14    468.46
15    530.86
16    598.42
17    671.38
18    749.98
19    834.49
20    925.13

Wire Loading Model Mode: enclosed.

Timing Ranges:

    No timing ranges specified.

Pin Input Delays:

    None specified.

Pin Output Delays:

    None specified.

Disabled Timing Arcs:

    No arcs disabled.

Required Licenses:

    None Required

Design Parameters:

    None specified.
1
dc_shell>
```

Quality Of Reports / QOR Report:

```
dc_shell> report_qor
```

```
*****
Report : qor
Design : top
Version: W-2024.09-SP3
Date   : Fri Aug 7 12:18:40 2026
*****
```

Timing Path Group 'clk'

```
-----
Levels of Logic:           6.00
Critical Path Length:      0.54
Critical Path Slack:       6.89
Critical Path Clk Period: 10.00
Total Negative Slack:      0.00
No. of Violating Paths:    0.00
Worst Hold Violation:      0.00
Total Hold Violation:      0.00
No. of Hold Violations:    0.00
-----
```

Cell Count

```
-----
Hierarchical Cell Count:   5
Hierarchical Port Count:  79
Leaf Cell Count:           310
Buf/Inv Cell Count:        36
Buf Cell Count:            0
Inv Cell Count:            36
CT Buf/Inv Cell Count:     0
Combinational Cell Count:  245
Sequential Cell Count:     65
Macro Count:               0
-----
```

Area

```
-----
Combinational Area:        531.923399
Noncombinational Area:    430.011662
Buf/Inv Area:              45.745920
Total Buffer Area:         0.00
Total Inverter Area:       45.75
Macro/Black Box Area:      0.000000
Net Area:                  160.869217
-----
Cell Area:                 961.935061
Design Area:               1122.804278
-----
```

Design Rules

```
-----
Total Number of Nets:      366
Nets With Violations:      0
Max Trans Violations:      0
Max Cap Violations:        0
-----
```

Hostname: taradash.soa.ac.in

Compile CPU Statistics

```
-----
Resource Sharing:           0.36
Logic Optimization:         0.38
Mapping Optimization:       0.40
-----
Overall Compile Time:       3.48
Overall Compile Wall Clock Time: 3.85
-----
```

Design WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0

Design (Hold) WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0

UVM Terminal Output:

```
: uvm_test_top.env.sbd [SCBD_MATCH] Match! Expected: 2d, Actual: 2d
!17125000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
): uvm_test_top.env.sbd [SCBD_REPORT] =====
): uvm_test_top.env.sbd [SCBD_REPORT]                      UART SCOREBOARD RESULTS
): uvm_test_top.env.sbd [SCBD_REPORT] =====
): uvm_test_top.env.sbd [SCBD_REPORT] Total Test Cases Passed : 9
): uvm_test_top.env.sbd [SCBD_REPORT] Total Test Cases Failed : 0
): uvm_test_top.env.sbd [SCBD_REPORT] =====
): uvm_test_top.env.sbd [SCBD_PASS] Test Passed: All data matched perfectly.
!17125000: reporter [UVM/REPORT/SERVER]
```

7. Static Timing Analysis reports

```

*****
Report : timing
        -path full
        -delay max
        -max_paths 1
Design : top
Version: W-2024.09-SP3
Date   : Fri Aug  7 12:05:13 2026
*****

Operating Conditions: tt0p85v25c   Library: saed32rvt_tt0p85v25c
Wire Load Model Mode: enclosed

Startpoint: rst (input port clocked by clk)
Endpoint: TX/state_reg[0]
          (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

Des/Clust/Port      Wire Load Model      Library
-----
top                 8000                 saed32rvt_tt0p85v25c
Transmitter         ForQA                 saed32rvt_tt0p85v25c

Point              Incr              Path
-----
clock clk (rise edge)          0.00              0.00
clock network delay (ideal)     0.00              0.00
input external delay            2.50              2.50 f
rst (in)                      0.02              2.52 f
TX/rst (Transmitter)           0.00              2.52 f
TX/U10/Y (INVX1_RVT)           0.07              2.59 r
TX/U33/Y (NAND2X0_RVT)         0.07              2.65 f
TX/U32/Y (AOI21X1_RVT)        0.10              2.75 r
TX/U31/Y (NAND3X0_RVT)        0.09              2.84 f
TX/U30/Y (NAND3X0_RVT)        0.10              2.94 r
TX/U25/Y (NAND4X0_RVT)        0.09              3.03 f
TX/state_reg[0]/D (DFFX1_RVT) 0.01              3.04 f
data arrival time                                     3.04

clock clk (rise edge)          10.00             10.00
clock network delay (ideal)     0.00             10.00
TX/state_reg[0]/CLK (DFFX1_RVT) 0.00             10.00 r
library setup time              -0.07             9.93
data required time               9.93

-----
data required time               9.93
data arrival time               -3.04
-----
slack (MET)                      6.89

```

8. **Conclusions**

This project successfully demonstrates the end-to-end RTL design, loopback verification, and ASIC logic synthesis of a Universal Asynchronous Receiver-Transmitter (UART) architecture. Key learning outcomes were achieved in modular Verilog HDL design, specifically in structuring finite state machines (FSMs) for serial data serialization and deserialization, parameterized baud tick generation, and noise-tolerant asynchronous communication. A critical technical insight was the implementation of a 16x oversampling mechanism with midpoint sampling at sample 7, which provides robust glitch rejection and reliable frame capture despite potential clock drifts between communicating devices. The entire UART was implemented, debugged and verified using Synopsys VCS and Verdi. Furthermore, the ASIC synthesis workflow using Synopsys Design Compiler with the 32nm SAED32 standard cell library offered practical exposure to TCL constraint scripting, timing closure, and gate-level netlist generation. The resulting static timing analysis confirmed setup timing closure with a positive slack of 6.89 ns under a 10 ns clock period (zero violating paths), alongside a total cell area of 1122.80 μm^2 and a total power dissipation of 58.20 μW , validating the efficiency and hardware feasibility of the design.

Building upon this baseline UART module, several future extensions can be explored to expand its functionality for complex embedded systems and System-on-Chip (SoC) integrations. The core architecture can be enhanced by incorporating configurable parity bit generation and checking (even/odd) to enable hardware-level error detection, alongside support for multi-bit stop configurations and variable data payload lengths. To reduce CPU interrupt frequency during high-bandwidth communications, dual hardware FIFO buffers (such as 16-byte RX/TX queues) can be integrated directly into the communication pipeline. Furthermore, wrapping the module with standard bus protocol interfaces—such as the AMBA APB or AXI-Lite bus—will enable seamless integration as an IP peripheral within microprocessor sub-systems, while implementing dynamic clock-gating techniques can further optimize switching power during idle transmission states.